

Requirements Writing and Review Guide

Document ID: GKB-REQ-QUALITY-002

Version: 1.0

Scope: General Knowledge Base

Intended use: reusable requirement quality guidance

Owner: Systems Engineering Office

1. Purpose

This guide defines how to write and review requirements so they can be tested, traced, and maintained.

It is based on common systems-engineering practice, including ISO/IEC/IEEE 29148-style requirement engineering principles and INCOSE guidance on well-formed needs and requirements.

2. What Is a Requirement?

A requirement is a statement that expresses a necessary capability, constraint, quality, interface, or condition that a system shall satisfy.

In test planning, a requirement must be specific enough to drive verification.

3. Requirement Statement Pattern

A practical requirement pattern is:

[Subject] shall [required behavior] [measurable condition or constraint].

Examples:

- The gateway shall reject unauthenticated API requests with HTTP 401.
- The system shall process 2,000 telemetry records per minute under nominal load.
- The application shall retain audit events for at least 180 days.

4. Characteristics of Good Requirements

4.1 Necessary

The requirement supports a real stakeholder need, risk control, regulatory need, or architectural decision.

Review question:

- What problem does this requirement solve?

4.2 Appropriate

The requirement belongs at the correct level of abstraction.

Bad:

- The enterprise system shall use a blue button on the login page.

Better:

- The system shall provide an authenticated login mechanism.

4.3 Singular

The requirement expresses one thing.

Bad:

- The system shall authenticate users and export reports and send alerts.

Better:

- The system shall authenticate users before granting access.
- The system shall export reports in PDF format.
- The system shall send alerts when thresholds are exceeded.

4.4 Unambiguous

The requirement has only one reasonable interpretation.

Avoid:

- fast;
- user-friendly;
- robust;
- as soon as possible;
- appropriate;
- sufficient.

Better:

- within 5 seconds;
- available to users with the administrator role;
- retry up to 3 times with exponential backoff.

4.5 Complete

The requirement contains enough information to design and verify the behavior.

Incomplete:

- The system shall send alerts.

Complete:

- The system shall send an alert to the site administrator when buffer usage exceeds 80 percent for more than 60 seconds.

4.6 Feasible

The requirement can be implemented within known technology, cost, schedule, and operational constraints.

Review question:

- Is this possible on the target platform?

4.7 Verifiable

The requirement can be checked by inspection, analysis, demonstration, or test.

Bad:

- The system shall be reliable.

Better:

- The system shall operate continuously for 30 days under nominal load without manual restart.

4.8 Traceable

The requirement has an identifier and can be linked to:

- source document;
- stakeholder need;
- design element;

- test case;
- defect;
- release decision.

5. Requirement Types

Type	Description	Verification approach
Functional	System behavior or capability	Functional/API/system test
Performance	Timing, throughput, capacity	Measurement under load
Reliability	Availability, recovery, endurance	Fault injection/endurance test
Security	Authentication, authorization, confidentiality	Security test/negative test
Interface	Protocol, API, data exchange	Contract and compatibility test
Safety	Hazard control or safety behavior	Safety analysis and evidence test
Usability	Human task success	User task observation
Regulatory	Compliance obligation	Inspection, audit, evidence review

6. Requirement Attributes

A requirement record should include:

- unique ID;
- title;
- statement;
- type;
- priority;
- source;
- owner;
- rationale;
- acceptance criteria or verification method;
- status;
- version or baseline;
- linked test cases.

7. Writing Rules

Rule 1: Use Shall for Binding Requirements

Use **shall** for mandatory requirements.

Use **should** for recommendations only if the organization distinguishes recommendations from requirements.

Rule 2: Avoid Multiple Requirements in One Sentence

Split requirements joined by:

- and;
- or;
- as well as;
- including;
- while also.

Rule 3: Include Measurable Thresholds

If the requirement concerns timing, capacity, accuracy, or retention, include numbers.

Examples:

- within 30 seconds;
- at least 24 hours;
- no more than 1 percent packet loss;
- retain for 180 days.

Rule 4: State the Actor or Subject

Bad:

- Must be logged.

Better:

- The system shall log every failed login attempt.

Rule 5: Define Failure Behavior

Good requirements describe what happens when something fails.

Example:

- If the cloud endpoint is unavailable, the gateway shall buffer telemetry locally.

8. Requirement Review Checklist

For each requirement, ask:

- Is it necessary?
- Is it singular?
- Is it unambiguous?
- Is it complete?
- Is it feasible?
- Is it verifiable?
- Does it have a source?
- Does it have a unique ID?
- Does it have priority?
- Does it have an acceptance hint or verification method?

9. Defect Patterns in Requirements

9.1 Vague Requirement

Example:

- The dashboard shall be fast.

Correction:

- The dashboard shall load the project overview in less than 3 seconds for projects with up to 1,000 requirements.

9.2 Non-Testable Requirement

Example:

- The system shall provide a world-class experience.

Correction:

- The user shall be able to create a project and upload a document in fewer than 5 steps.

9.3 Compound Requirement

Example:

- The system shall authenticate users and encrypt logs.

Correction:

- The system shall authenticate users before granting access.
- The system shall encrypt logs at rest.

9.4 Missing Trigger

Example:

- The system shall send an alert.

Correction:

- The system shall send an alert when CPU usage exceeds 85 percent for more than 5 minutes.

9.5 Missing Tolerance

Example:

- The sensor shall report accurate temperature.

Correction:

- The sensor shall report temperature within plus or minus 0.5 degrees Celsius over the range 0 to 80 degrees Celsius.

10. Guidance for AI Extraction

When AI extracts requirements from documents, it should:

- preserve original IDs when available;
- distinguish requirements from background information;
- not convert every sentence into a requirement;
- avoid inventing thresholds;
- keep source citations;
- flag uncertainty;
- split compound statements;
- classify requirement type;
- include acceptance hints.

11. References

- ISO/IEC/IEEE 29148 systems and software engineering requirements guidance.
- INCOSE guidance on well-formed needs and requirements.
- NASA systems-engineering practice on verification planning and traceability.